

Spring Bean Lifecycle & Scopes

A Beginner-Friendly Guide

Chapter 03 | Sub-Chapter 03-02

Bean Scopes	Singleton, Prototype, Request, Session, Application
Bean Lifecycle Phases	Instantiation to Destroy - 8 key phases
@PostConstruct / @PreDestroy	Annotation-based init and cleanup
InitializingBean / DisposableBean	Interface-based lifecycle hooks
BeanPostProcessor	Customize beans after creation
@Lazy Initialization	Delay bean creation until first use

1 Bean Scopes

Singleton, Prototype, Request, Session, Application

Spring supports five main bean scopes. The scope tells Spring how many instances of a bean to create and how long each instance lives.

Scope Quick Reference

Scope	Instances	Lifetime	Typical Use
singleton	One per container	Whole app	Services, DAOs
prototype	New per request	Until discarded	Stateful objects
request	One per HTTP request	HTTP request	Web controllers
session	One per HTTP session	HTTP session	User session data
application	One per ServletContext	App lifetime	App-wide config

Java Example - Declaring Scopes

```
import org.springframework.context.annotation.Scope;
import org.springframework.stereotype.Component;

@Component
@Scope("singleton") // default - one shared instance
public class UserService { }

@Component
@Scope("prototype") // new instance each time
public class ShoppingCart { }

@Component
@Scope("request") // one per HTTP request
public class RequestLogger { }
```

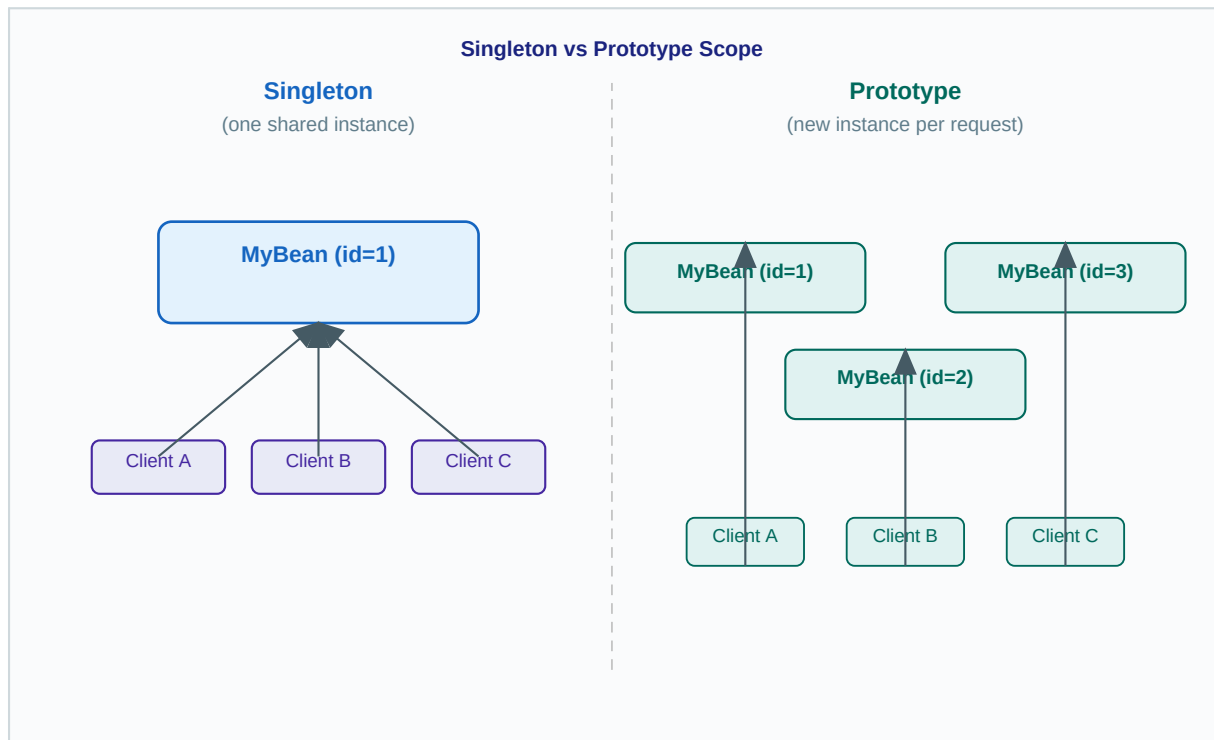


Figure 1 - Singleton shares one instance; Prototype creates a fresh one for each caller.

Key Rules

- singleton is the default scope - Spring returns the same bean instance on every `getBean()` call.
- prototype beans are NOT destroyed by Spring - the caller is responsible for cleanup.
- request, session, and application scopes require a web-aware `ApplicationContext`.
- Use `@Scope(value='prototype', proxyMode=ScopedProxyMode.TARGET_CLASS)` to inject prototype into singleton.
- Never store mutable request-specific state in a singleton bean.

Q1. What is the default scope of a Spring bean?

singleton - Spring creates exactly one instance per `ApplicationContext` and reuses it.

Q2. When should you use prototype scope?

When each caller needs its own independent copy with separate state, such as a shopping cart.

Q3. Can request-scoped beans be used outside a web context?

No - request, session, and application scopes require a web-aware `ApplicationContext` (e.g., `DispatcherServlet`).

Q4. How do you inject a prototype bean into a singleton?

Use `proxyMode = ScopedProxyMode.TARGET_CLASS` in `@Scope` so Spring creates a proxy that fetches a fresh instance each time.

Q5. What happens to a prototype bean after Spring hands it out?

Spring forgets it - lifecycle callbacks like `@PreDestroy` are NOT called. The caller must manage cleanup.

2 Bean Lifecycle Phases

Instantiation to Destroy - 8 key stages

Spring manages every singleton bean through a well-defined lifecycle. Understanding the phases helps you hook into the right moment.

The 8 Lifecycle Phases



Figure 2 - Bean lifecycle from object creation to container shutdown.

Phase-by-Phase Explanation

Phase	What Happens
1. Instantiation	Spring calls the constructor (or factory method) to create the raw object.
2. Populate Properties	Spring injects @Autowired fields and setter-injected values.

3. BeanNameAware	Spring calls setBeanName() if the bean implements BeanNameAware.
4. BeanFactoryAware	Spring calls setBeanFactory() if the bean implements BeanFactoryAware.
5. @PostConstruct	Your @PostConstruct method runs - use for validation / cache warm-up.
6. Bean Ready	Bean is fully initialized and returned to the application for use.
7. @PreDestroy	Your @PreDestroy method runs just before the container shuts down.
8. Destroy	Spring releases the bean; resources are freed.

Java Example - Lifecycle Hooks in One Class

```
import jakarta.annotation.PostConstruct;
import jakarta.annotation.PreDestroy;
import org.springframework.beans.factory.BeanNameAware;
import org.springframework.stereotype.Component;

@Component
public class DatabasePool implements BeanNameAware {

    @Override
    public void setBeanName(String name) {
        System.out.println("Bean name: " + name);    // Phase 3
    }

    @PostConstruct
    public void init() {
        System.out.println("Opening DB pool...");    // Phase 5
    }

    @PreDestroy
    public void cleanup() {
        System.out.println("Closing DB pool...");    // Phase 7
    }
}
```

Key Rules

- Phases 3-4 (Aware interfaces) run before @PostConstruct.
- BeanPostProcessor methods wrap around @PostConstruct (before and after).
- prototype-scoped beans skip phases 7 and 8 - Spring never calls @PreDestroy on them.
- The lifecycle applies per ApplicationContext - child contexts have their own lifecycle.
- Always close the context with context.close() to trigger @PreDestroy in standalone apps.

Q1. In what order do @PostConstruct and property injection run?

Property injection (Phase 2) always runs BEFORE @PostConstruct (Phase 5).

Q2. Why might @PreDestroy never be called?

If the ApplicationContext is not closed properly, or the bean is prototype-scoped.

Q3. What is BeanNameAware used for?

It lets a bean know its own bean ID - useful for logging or dynamic lookups.

Q4. Does every bean go through all 8 phases?

Phases 3 and 4 only apply if the bean implements those Aware interfaces. Others are optional.

Q5. How do you trigger the destroy phase in a standalone Spring app?

Call `((ConfigurableApplicationContext) ctx).close()` or register a shutdown hook with `ctx.registerShutdownHook()`.

3 **@PostConstruct and @PreDestroy**

Annotation-based init and cleanup callbacks

These two annotations are the simplest and most modern way to add lifecycle behaviour to a Spring bean. They come from the jakarta.annotation package (javax.annotation before Java EE 8).

Java Example - Cache Manager

```
import jakarta.annotation.PostConstruct;
import jakarta.annotation.PreDestroy;
import org.springframework.stereotype.Component;
import java.util.HashMap;
import java.util.Map;

@Component
public class CacheManager {

    private Map<String, Object> cache;

    @PostConstruct          // called once - after injection is done
    public void initCache() {
        cache = new HashMap<>();
        System.out.println("Cache initialized.");
    }

    @PreDestroy             // called once - just before bean is destroyed
    public void clearCache() {
        cache.clear();
        System.out.println("Cache cleared on shutdown.");
    }
}
```

Key Rules

- @PostConstruct method must be void, take no parameters, and not be static.
- @PreDestroy method must be void, take no parameters, and not be static.
- Both annotations are part of JSR-250, not Spring itself - they work in any Java EE / Jakarta EE container.
- Add the jakarta.annotation-api dependency if it is not on the classpath (Spring Boot includes it).
- @PreDestroy is NOT called for prototype beans - only for singleton (and other non-prototype) beans.

Q1. What is @PostConstruct used for?

Running initialization logic after Spring has injected all dependencies - e.g., loading config, warming caches.

Q2. Can @PostConstruct throw a checked exception?

Yes - Spring will wrap and propagate it, preventing the bean from becoming ready.

Q3. What is the difference between a constructor and @PostConstruct?

The constructor runs before injection; @PostConstruct runs after all fields are injected, so you can safely use them.

Q4. Which annotation handles resource cleanup before shutdown?

@PreDestroy - it runs just before the bean is removed from the container.

Q5. Are these annotations Spring-specific?

No - they are standard Java annotations (JSR-250). Spring simply honors them as part of its lifecycle processing.

4 InitializingBean / DisposableBean

Interface-based lifecycle contracts

Before annotations existed, Spring provided two interfaces for lifecycle callbacks. They are still supported but annotations are now preferred.

Java Example - Using the Interfaces

```
import org.springframework.beans.factory.DisposableBean;
import org.springframework.beans.factory.InitializingBean;
import org.springframework.stereotype.Component;

@Component
public class FileProcessor
    implements InitializingBean, DisposableBean {

    // Called after all properties are set (like @PostConstruct)
    @Override
    public void afterPropertiesSet() throws Exception {
        System.out.println("FileProcessor ready.");
    }

    // Called before the bean is destroyed (like @PreDestroy)
    @Override
    public void destroy() throws Exception {
        System.out.println("FileProcessor shutting down.");
    }
}
```

Interface vs Annotation Comparison

Feature	Interface approach	Annotation approach
Method name	Fixed (afterPropertiesSet / destroy)	Any name you choose
Coupling	Couples bean to Spring API	No Spring dependency
Testing	Must have Spring context	Can call method directly
Recommended?	Legacy code only	Yes - preferred in modern apps

Key Rules

- InitializingBean.afterPropertiesSet() is equivalent to @PostConstruct - both run at Phase 5.
- DisposableBean.destroy() is equivalent to @PreDestroy - both run at Phase 7.
- If both interface and annotation are present, annotation runs first.
- Prefer annotations in new code to avoid Spring API coupling in your domain classes.
- You can also specify init-method / destroy-method in @Bean or XML as a third alternative.

Q1. What interface provides an init callback in Spring?

InitializingBean - its afterPropertiesSet() method runs after property injection, just like @PostConstruct.

Q2. What is the modern alternative to implementing DisposableBean?

@PreDestroy annotation - it is cleaner and does not tie your class to the Spring API.

Q3. Which runs first if a bean has both @PostConstruct and afterPropertiesSet()?

@PostConstruct runs first, then afterPropertiesSet().

Q4. How do you set init / destroy methods without annotations or interfaces?

Use @Bean(initMethod='start', destroyMethod='stop') or the equivalent XML attributes.

Q5. Why avoid InitializingBean in new projects?

It couples your class to org.springframework.beans.factory - making unit testing harder.

5 BeanPostProcessor

Intercept and customize beans after creation

BeanPostProcessor is a Spring extension point that lets you intercept every bean during its lifecycle - right before and right after initialization. Spring itself uses it internally for @Autowired, @PostConstruct, and AOP proxies.

How It Fits in the Lifecycle

Callback	When it runs
postProcessBeforeInitialization()	After property injection, BEFORE @PostConstruct
postProcessAfterInitialization()	AFTER @PostConstruct - bean is now fully ready

Java Example - Logging BeanPostProcessor

```
import org.springframework.beans.BeansException;
import org.springframework.beans.factory.config.BeanPostProcessor;
import org.springframework.stereotype.Component;

@Component
public class LoggingPostProcessor implements BeanPostProcessor {

    @Override
    public Object postProcessBeforeInitialization(
        Object bean, String beanName) throws BeansException {
        System.out.println("Before init: " + beanName);
        return bean;    // must return the bean (or a wrapper)
    }

    @Override
    public Object postProcessAfterInitialization(
        Object bean, String beanName) throws BeansException {
        System.out.println("After init: " + beanName);
        return bean;
    }
}
```

Key Rules

- BeanPostProcessor applies to ALL beans in the context - use beanName checks to target specific beans.
- Always return the bean (or a proxy) - returning null will break the context.
- BeanPostProcessor beans are instantiated very early, before other beans.
- Spring's @Autowired processing is done by AutowiredAnnotationBeanPostProcessor.
- You can return a different object (e.g., a proxy) from postProcessAfterInitialization to wrap the bean.

Q1. What is the purpose of BeanPostProcessor?

To apply cross-cutting logic (logging, validation, proxying) to all beans automatically, without modifying each bean class.

Q2. What happens if postProcessAfterInitialization returns null?

Spring throws a NullPointerException and the application context fails to start.

Q3. How does Spring's AOP work with BeanPostProcessor?

Spring's AOP proxy creator is itself a BeanPostProcessor that wraps beans in proxies after initialization.

Q4. Can you limit a BeanPostProcessor to specific beans?

Yes - check beanName or instanceof inside the method and skip beans you do not want to process.

Q5. When is a BeanPostProcessor bean itself initialized?

Before all other beans - Spring gives BeanPostProcessor beans priority during context startup.

6 @Lazy Initialization

Delay bean creation until first use

By default, Spring creates all singleton beans at startup (eager initialization). @Lazy tells Spring to wait and only create the bean when it is first requested.

Eager vs Lazy Comparison

Aspect	Eager (default)	Lazy (@Lazy)
Created at	Application startup	First getBean() or injection
Startup time	Slower (all beans)	Faster (fewer beans upfront)
Error discovery	At startup	At first use - harder to debug
Memory use	All beans loaded	Only used beans loaded
Best for	Most production beans	Rarely used or heavy beans

Java Example - @Lazy on a Bean

```
import org.springframework.context.annotation.Lazy;
import org.springframework.stereotype.Component;

@Component
@Lazy // only created when first injected or requested
public class HeavyReportGenerator {

    public HeavyReportGenerator() {
        // This line prints only when the bean is first needed
        System.out.println("HeavyReportGenerator created.");
    }

    public void generate() {
        System.out.println("Generating report...");
    }
}
```

Java Example - Enabling Global Lazy Initialization

```
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class MyApp {
    public static void main(String[] args) {
        SpringApplication app = new SpringApplication(MyApp.class);
        app.setLazyInitialization(true); // all beans are lazy
        app.run(args);
    }
}
```

Key Rules

- @Lazy on a @Component delays creation until the bean is first injected or fetched via getBean().
- Global lazy mode (setLazyInitialization(true)) speeds up Spring Boot startup in development.

- Errors in lazy beans are not detected until first use - avoid lazy for critical infrastructure beans.
- @Lazy on an @Autowired injection point forces a proxy even if the bean itself is not @Lazy.
- Combine @Lazy with @Scope('prototype') carefully - each lazy prototype call creates a new instance.

Q1. What does @Lazy do to a singleton bean?

It prevents Spring from creating the bean at startup; instead, it is created on the first injection or getBean() call.

Q2. What is the main risk of lazy initialization?

Configuration errors or missing dependencies are only discovered at first use, not at startup.

Q3. How do you make ALL beans lazy in a Spring Boot app?

Call SpringApplication.setLazyInitialization(true) before running the app.

Q4. Can you put @Lazy on an @Autowired field?

Yes - Spring injects a proxy. The real bean is not created until a method on the proxy is called.

Q5. When is @Lazy most beneficial?

For heavy, rarely used beans (report generators, batch processors) that you do not want loaded at every startup.